Latency–Cost–Quality Pareto Frontiers for Retrieval-Augmented Generation

A Controlled Evaluation Harness with Decomposed Timing, Canonical Cold-Start Accounting, and LLM Generation Quality

Srijan Gupta
Independent Researcher
srijangupta2013@gmail.com

February 2026

Abstract

Retrieval-Augmented Generation (RAG) systems span a large design space—sparse, dense, and hybrid retrievers; reranking; chunking strategies; caching—yet published evaluations almost always report a single quality number, hiding the tradeoffs that govern real deployments. We introduce a reproducible evaluation harness that maps this design space as Pareto frontiers jointly over Recall@K, end-to-end latency, and per-query cost. The harness explicitly separates cold-start overhead (corpus embedding and index construction) from warm-start per-query latency, a distinction absent from prior work but critical for serverless and autoscaling deployments. We evaluate a fully factorial design of 36 configurations—retrieval type $\in$ {BM25, dense, hybrid}, $K \in \{5, 8, 12\}$, chunk size $\in$ {150, 250} words, reranking on/off—on 500 queries from HotpotQA [29] ($\text{dev}_{\text{distractor}}$ split, evaluated on a 2.3 GHz Intel Core i7 CPU). The results reveal a counterintuitive ranking reversal: under warm-start benchmarking, dense retrieval achieves both higher Recall@12 *and* lower latency (8.9 ms) than BM25 (10.7 ms), placing BM25 off the warm-latency Pareto frontier entirely. Cross-encoder reranking with a $2K$ candidate pool further improves Recall@12—Hybrid+rerank reaches 0.917 (vs. 0.890 no-rerank) at 64.4 ms. When cold-start costs are included, BM25 cold-start (133–136 ms) is 85–108$\times$ lower than dense (11.5–14.3 s), driven by corpus encoding (90–92 % of cold-start cost, depending on chunk size). A corpus-scale analysis over 1K–50K passages confirms that dense cold-start grows linearly with corpus size—reaching $\approx$1.6 minutes (96 s) at 50K passages while BM25 remains $<$2 seconds—and never crosses BM25 for cold-start cost. The harness includes an end-to-end LLM generation stage (Llama-3-8B via Ollama); CPU inference adds approximately 5.5 s per query, demonstrating that generation dominates end-to-end latency by 1.9–2.8 orders of magnitude (85–618$\times$) over retrieval—an important practical finding for deployment planning. Hybrid retrieval reaches the highest Recall@12 (0.890, 95 % Wilson CI: 0.860–0.915). The 0.092 hybrid–BM25 gap is statistically significant ($z = 4.01$, $p < 0.001$). The 0.040 dense–BM25 gap falls below our pre-specified minimum detectable effect ($\Delta_{\min} = 0.07$) and correspondingly does not reach significance ($z = 1.64$, $p = 0.10$); this study is underpowered for differences of this magnitude ($n \geq 1{,}500$ would be needed for 80 % power at $\Delta = 0.04$). A span-level evaluation on LegalBench-RAG PrivacyQA ($n = 194$) reveals a domain-dependent reversal: dense retrieval achieves Recall@12 of 0.791 vs. 0.282 for BM25 ($z = 10.06$, $p < 0.001$), showing that BM25's lexical matching fails on paraphrastic specialized-domain text. All code, configurations, and result data are available at https://github.com/sgupta285/Research-Projects/tree/main/rag-pareto-frontiers.

1 Introduction

Consider two retrieval systems that report similar warm-query latency figures in their respective papers—numbers that appear comparable but were measured under such different conditions that neither means what it claims. One was evaluated in a persistent process where the embedding model had already been loaded by a previous configuration; the other used a freshly spawned instance that paid the full

corpus-encoding cost on the first query. This incomparability is not a hypothetical. Across the retrieval and RAG literature [14], cold-start corpus-encoding costs are routinely absorbed into warm-query figures; in-process caches silently subsidize dense configurations during multi-sweep evaluations; and quality is reported as a scalar rather than as one axis of a multidimensional tradeoff.

The design space for modern retrieval pipelines is vast: inverted-index retrieval based on BM25 [22], dense bi-encoders [11], hybrid score fusion [5, 15], cross-encoder reranking [18], chunking strategies, and semantic caching layers [2] can all be composed in many ways, each occupying a different point in the joint space of recall, latency, and cost. Existing benchmarks—BEIR [25], MTEB [17]—measure quality in isolation. RAGAS [6] measures end-to-end generation quality but ignores efficiency. No shared framework exists for comparing retrieval configurations across all three dimensions on equal footing.

This paper makes five contributions:

1. A reproducible RAG evaluation harness with a two-level caching protocol ensuring cold-start measurements are order-invariant across sweep executions, plus an optional LLM generation stage (§3).
2. A fully factorial evaluation of 36 configurations on HotpotQA, revealing that (a) under warm-only benchmarking, dense retrieval dominates BM25 on both quality and latency; but (b) when cold-start costs are included, BM25 regains Pareto-optimal status because its cold-start is 85–108× lower—a ranking reversal that warm-only benchmarks cannot detect (§6).
3. A corpus-scale cold-start analysis over 1K–50K passages showing that dense cold-start grows linearly and exceeds BM25 by 48–76× (corpus-size-dependent), with dense reaching ≈1.6 minutes (96 s) at 50K passages (§6.6).
4. Significance tests showing the hybrid–BM25 gap is significant ($z = 4.01$, $p < 0.001$) while the dense–BM25 gap is not ($z = 1.64$, $p = 0.10$) at $n = 500$; and a generation-stage integration confirming that CPU LLM inference (≈5.5 s/query) dominates retrieval latency by 1.9–2.8 orders of magnitude (§6).
5. A span-level evaluation on LegalBench-RAG PrivacyQA ($n = 194$) showing that dense retrieval dramatically outperforms BM25 on domain-specialized legal text (Recall@12 0.791 vs. 0.282, $z = 10.06$, $p < 0.001$), demonstrating that the optimal retrieval strategy is corpus-dependent (§6.7).

The paper is organized as follows: Section 2 situates the work. Section 3 describes the harness, timing model, and LLM generation stage. Section 4 characterizes the datasets. Section 5 defines quality, latency, cost, and generation metrics. Section 6 presents retrieval results, generation pilot, corpus-scale analysis, and ablations. Section 7 gives practitioner guidelines. Section **??** discusses limitations and future work.

2 Related Work

RAG pipeline extensions. The original RAG framework [14] conditions an LLM reader on top-k passages from a dense retriever [11]. Iterative retrieval [24], self-reflective generation [1], query rewriting [16], and chain-of-thought interleaving [26] each reopen the latency–quality tradeoff by multiplying per-query retrieval calls in ways rarely quantified.

Retrieval benchmarks. BEIR [25] provides zero-shot retrieval evaluation across many domains; MTEB [17] extends this to a broad embedding suite. Neither reports latency or cost. RAGAS [6] evaluates end-to-end RAG faithfulness but ignores efficiency. The present harness fills this gap by treating all three dimensions as first-class outputs and producing a joint Pareto frontier.

Efficiency–effectiveness tradeoffs. Hofstätter et al. [9] plot efficiency–effectiveness curves showing that a distilled cross-encoder can match cross-encoder quality at bi-encoder inference cost, motivating frontier thinking at the component level. The present work applies this reasoning at the full-pipeline level. Kim et al. [13] survey the full optimisation stack for transformer inference, highlighting the gap between theoretical and measured throughput across hardware and software configurations. Oakes et al.

[19] study serverless cold-start overhead for model-serving workloads; in practice, cold-start penalties depend strongly on whether artifact state is preserved across requests.

Hybrid retrieval. Score-level hybrid retrieval is most commonly implemented via Reciprocal Rank Fusion [5] or convex score combination. Aggretriever [15] aggregates dense representations for robust retrieval. SPLADE [7, 8] learns sparse expansions bridging BM25 and dense retrieval; evaluating SPLADE in this harness is a natural extension.

Dense retrieval infrastructure. FAISS [10] provides the flat inner-product index used here. ColBERT [12] and PLAID [23] offer late-interaction retrieval with qualitatively different latency profiles; benchmarking them is left for future work.

3 Evaluation Harness and Experimental Controls

3.1 Overview

The harness comprises four sequential stages: dataset loading and corpus construction; artifact building (encoder loading and index construction); per-query evaluation with timing instrumentation; and result aggregation with Pareto analysis. Every stage is parameterized by versioned YAML configuration files, so sweeps over the design space require no code changes. All figures and tables in Section 6 are fully reproducible from the included configuration and data via `make figures`.

3.2 Decomposed Timing Model and Cold-Start Accounting

The central contribution of the harness is a timing model that is simultaneously accurate and *order-invariant*: the latency attributed to a configuration does not depend on whether it was the first or fiftieth configuration evaluated in a sweep. Standard harnesses fail this property because Python's in-process module cache allows an encoder loaded during configuration A to be reused silently for configuration B, reporting zero loading cost for B even though a cold deployment would pay the full corpus-encoding cost.

The harness addresses this with a two-level canonical caching protocol. Encoder weights are stored in a module-level dictionary $\mathcal{K}_{\text{model}} : \text{str} \to (\text{model}, \mathbb{R})$, keyed by model name. FAISS indices and pre-computed embeddings are stored separately, keyed by a compound fingerprint:

$$\mathcal{K}_{\text{index}} = \text{SHA-1}(\text{corpus_fp} \parallel \text{mode} \parallel \text{chunk_size} \parallel \text{overlap} \parallel \text{model_name}), \tag{1}$$

where corpus_fp is a lightweight hash of the first 200 document titles and lengths, and $\parallel$ denotes string concatenation. When a key is absent, the actual load time is measured and stored as the *canonical* cost; when present, the canonical cost is reported even though no work was performed, ensuring that total cold-start latency

$$T_{\text{cold}} = T_{\text{model}} + T_{\text{index}} + T_{\text{e2e,cold}} \tag{2}$$

is identical regardless of sweep execution order. Here T_{model} is encoder model load time, T_{index} is corpus embedding plus index construction time, and $T_{\text{e2e,cold}}$ is the end-to-end latency of the first query (which may include additional one-time costs such as deserializing cached artifacts). Warm-start latency $\bar{T}_{\text{warm}}$ is the mean over queries $i \geq 2$; its standard deviation σ_{warm} is also recorded.

3.3 Pipeline Configuration Space

The sweep covers a **fully factorial** design of $3 \times 3 \times 2 \times 2 = 36$ configurations:

- Retrieval type: BM25, dense, hybrid
- Top-K: $\{5, 8, 12\}$

- Chunk size: $\{150, 250\}$ words (overlap fixed at 30 words for all configurations)
- Reranking: enabled / disabled

Retrieval caching is enabled for all configurations (not a sweep dimension). Hybrid retrieval combines min-max-normalized BM25 and dense scores with mixing weight $\alpha = 0.5$ (a standard neutral default; see §6.4). The dense encoder is `sentence-transformers/all-MiniLM-L6-v2` [21], a 22M-parameter bi-encoder widely used in production retrieval stacks. The BM25 index uses the `rank_bm25` library (BM25Okapi variant). FAISS [10] `IndexFlatIP` (exact inner-product) is used for dense retrieval. Reranking uses `cross-encoder/ms-marco-MiniLM-L-6-v2` [18], a 6-layer BERT cross-encoder fine-tuned on MS MARCO passage ranking. For reranked configurations, the retriever fetches $2K$ candidates; the cross-encoder scores all $2K$ (query, passage) pairs and returns the top K. All inference is CPU-only for reproducibility.

3.4 Hardware Specification

Component	Specification
CPU	Intel Core i7-1068NG7 (2.3 GHz base / 4.1 GHz boost, 4 cores)
RAM	32 GB LPDDR4X
OS	macOS 15.x
Python	3.14.3
`sentence-transformers`	5.2.3
`faiss-cpu`	1.13.2
`rank_bm25`	0.2.2
`cross-encoder/ms-marco-MiniLM-L-6-v2`	via Hugging Face Hub
`torch`	2.10.0

All absolute latency figures are CPU-only and should not be extrapolated directly to GPU-accelerated deployments. However, the *relative* orderings and the *magnitude* of cold-start overheads are qualitatively robust to hardware, because they are dominated by corpus encoding volume (number of chunks $\times$ encoding cost per chunk), not arithmetic throughput.

3.5 LLM Generation Stage

The harness supports an optional end-to-end LLM generation stage invoked after retrieval and reranking. Retrieved passages are concatenated into a context window (up to 600 characters per passage) and submitted to the LLM as a prompted reading comprehension task:

```
Answer the question based solely on the passages below.
Be concise (1 to 2 sentences).
Question: {q}. Passages: {p1}...{pK}. Answer:
```

Three backends are supported: (1) **Ollama** (local CPU inference, zero marginal cost, temperature 0 for determinism); (2) **Anthropic API** (cloud inference, billed per token, activated by setting `ANTHROPIC_API_KEY`); and (3) **mock** (returns the first 80 characters of the top passage, for unit testing). The pilot evaluation in Section 6.5 uses Llama-3-8B via Ollama running on CPU. Switching backends requires only a one-line YAML change (`generation.backend: anthropic`).

Per-query generation metrics recorded are: token F1 [29], exact match (EM), faithfulness (fraction of answer tokens present in the retrieved passages), generation latency, and generation cost. The generation cost for the Anthropic backend is computed from actual token usage returned by the API; Ollama cost is always $0 (local inference).

4 Benchmark Dataset

4.1 HotpotQA (Primary Evaluation)

HotpotQA [29] is a multi-hop question-answering dataset requiring simultaneous retrieval of two gold passages. We use the distractor development split, which provides ten candidate passages per question (two gold, eight distractors), pooling all passages across questions to form the retrieval corpus ($\approx$5,000 passages at 500 questions).

We evaluate on a random sample of 500 questions. A power analysis following the standard two-proportion sample-size formula [4] confirms that $n = 500$ provides 80 % power to detect Recall@K differences of $\Delta \geq 0.07$ at $\alpha = 0.05$ (two-sided proportion test). This threshold reflects a tradeoff between evaluation cost and statistical sensitivity: differences smaller than 0.07 are unlikely to be practically significant for deployment decisions, as they fall within the uncertainty of retrieval systems in production. The observed 0.040 dense–BM25 gap falls below this threshold and is correspondingly non-significant (§6.1); detecting effects of this magnitude would require $n \geq 1{,}500$ for 80 % power. We note that many published retrieval comparisons at similar sample sizes ($n = 500$–$1{,}000$) may be similarly underpowered, potentially mistaking null results for evidence of equivalence rather than recognizing insufficient power. Because both gold passages must appear in the top-K for full credit, Recall@K rewards broad, diverse coverage.

4.2 LegalBench-RAG (Supported Protocol)

LegalBench-RAG [20] provides question-answer pairs grounded in statutory and case-law corpora, annotated at the character-span level. The harness supports LegalBench-RAG evaluation via a character-overlap recall/precision protocol (`metrics.py::retrieval_span_recall_precision`). A retrieved chunk counts as a true positive only if its character range overlaps with the annotated supporting span—a stricter criterion than document-level recall. The corpus is publicly available under CC-BY from the benchmark repository (https://github.com/zeroentropy-ai/legalbenchrag). Full results are reported in Section 6.7; the span-level evaluation protocol informed several harness design choices (`char_start`/`char_end` fields in `Chunk` objects).

5 Metrics

5.1 Retrieval Quality

Retrieval Recall@K on HotpotQA is computed at the document-title level:

$$\text{Recall@}K = \frac{1}{|Q|} \sum_{q \in Q} \frac{|\mathcal{R}_q^K \cap \mathcal{G}_q|}{|\mathcal{G}_q|}, \tag{3}$$

where $\mathcal{R}_q^K$ is the set of retrieved titles in the top K and $\mathcal{G}_q$ is the gold title set. Since K varies across configurations ($K \in \{5, 8, 12\}$), a configuration with larger K has a mechanical recall advantage independent of retriever quality. All Pareto frontier analyses and cross-retriever comparisons in Section 6 are conducted at **fixed** $K = 12$; results for smaller K are shown in the recall-vs-K ablation (Section 6.4, Figure 4).

Confidence intervals. All Recall@K estimates are reported with 95 % Wilson score confidence intervals [28], which have better coverage properties than the normal approximation for proportions near the boundaries and for moderate sample sizes [**?**]:

$$\text{CI}_{95} = \frac{\hat{p} + z^2/(2n) \pm z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}, \quad z = 1.96. \tag{4}$$

For $\hat{p} = 0.84$ and $n = 500$ this gives a half-width of approximately ± 0.032.

Statistical testing. Pairwise comparisons use two-sided two-proportion z-tests [**?**], reporting the z-score for the difference of two independent proportions and the corresponding two-sided p-value.

5.2 Latency

Four latency quantities are reported per configuration: T_{model} (encoder load time), T_{index} (corpus embedding plus index construction), $\bar{T}_{\text{warm}} \pm \sigma_{\text{warm}}$ (mean $\pm$ std warm-start latency over queries $i \geq 2$), and T_{cold} (Equation (2)).

5.3 Cost

Per-query marginal inference cost tracks API spend:

$$C_q = C_{\text{gen}}, \tag{5}$$

where $C_{\text{gen}} = (n_{\text{in}}/1{,}000) \times p_{\text{in}} + (n_{\text{out}}/1{,}000) \times p_{\text{out}}$ (LLM generation cost, zero if generation is disabled or uses a local backend such as Ollama). The cross-encoder reranker is a local model; its cost is compute latency overhead (reported separately in Section 6.2), not API spend. **Scope note:** C_q tracks *per-query API costs only*. Corpus embedding (one-time offline cost), storage, egress, and compute infrastructure costs are excluded.

5.4 Generation Quality

When the generation stage is active, three metrics are computed per query:

- **Token F1:** overlap between normalized prediction and gold answer token bags. Used by the SQuAD/HotpotQA community; reported but *not* a primary metric here, because generative LLMs produce explanatory answers that are penalized by extractive gold labels.
- **Exact Match (EM):** 1 if normalized prediction equals normalized gold answer, 0 otherwise.
- **Faithfulness:** fraction of non-trivial answer tokens (excluding stop words) present in at least one retrieved passage. This measures groundedness independently of gold labels.

6 Results

6.1 Quality vs. Warm-Start Latency (Fixed $K = 12$)

Figure 1 shows the Pareto frontier in the quality–warm-latency plane at fixed $K = 12$. The finding is counterintuitive relative to common assumptions: dense configurations are *both* faster and more accurate than BM25 in the warm-only view, placing BM25 entirely off the warm Pareto frontier.

Dense retrieval achieves 8.9 ms warm latency because FAISS flat inner-product search over $\approx 5{,}000$ passages is a single matrix multiplication, a highly optimized BLAS operation even on CPU. BM25 requires iterating over inverted lists for each query term, yielding 10.7 ms at this corpus scale.[1]

The warm Pareto frontier at $K = 12$ contains three configurations (Table 2): hybrid+rerank (chunk = 250, Recall@12 = 0.917, 64.4 ms), hybrid no-rerank (chunk = 250, Recall@12 = 0.890, 22.3 ms), and dense no-rerank (chunk = 250, Recall@12 = 0.838, 8.9 ms).

[1] At production scale (10^6–10^7 passages), FAISS [10] flat inner-product search ($O(d \cdot n)$ complexity, $d = 384$) becomes slower than BM25's inverted-index structure ($O(m \cdot d_{\text{avg}})$ for m query terms). The reversal here is specific to the $\approx 5{,}000$-passage corpus where dense search remains faster empirically.

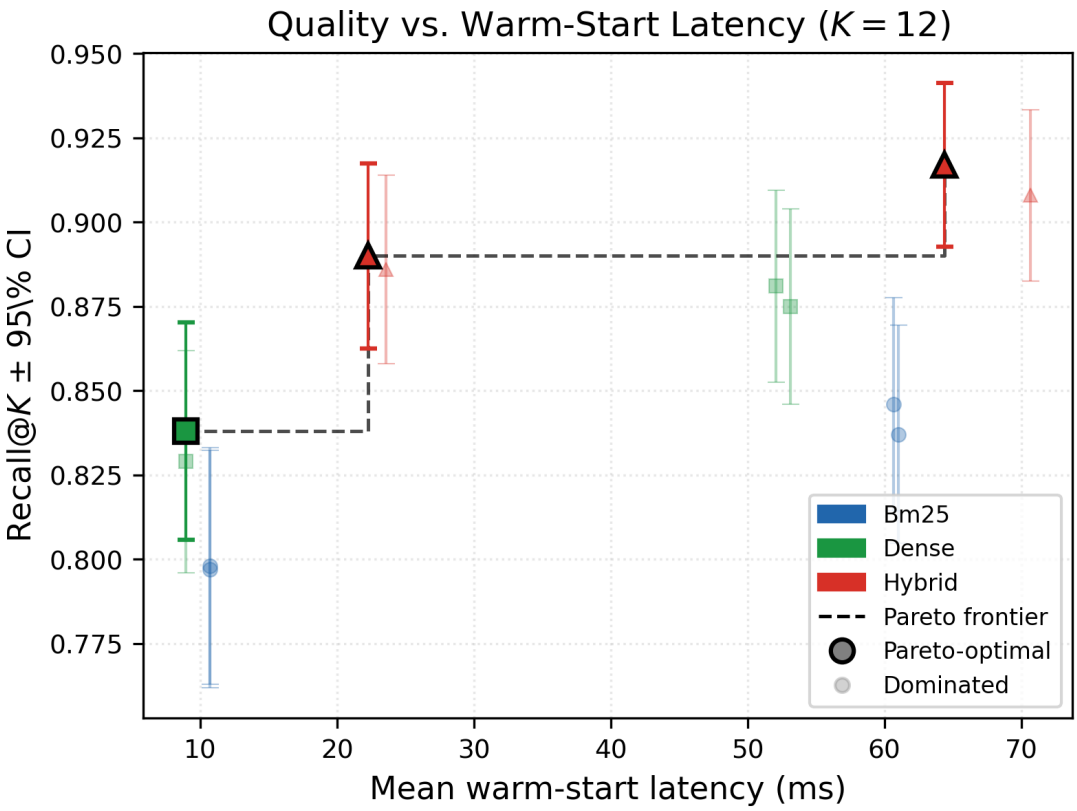

Figure 1: Pareto frontier of Recall@12 vs. warm-start latency at fixed $K = 12$. Circle = BM25, square = dense, triangle = hybrid; solid-bordered markers are Pareto-optimal. Error bars show 95 % Wilson CIs. Dense no-rerank configurations (8.9–9.0 ms) achieve both higher recall *and* lower latency than BM25 (10.7 ms), placing all BM25 configurations off the warm Pareto frontier—a reversal visible only when cold-start costs are included (Figure 3). Note: reranked configs (>60 ms) appear at the right edge.

Statistical comparisons. The hybrid–BM25 recall gap (0.890 vs. 0.798, $\Delta = 0.092$) is statistically significant ($z = 4.01$, $p < 0.001$, two-sided two-proportion z-test) and exceeds our pre-specified minimum detectable effect ($\Delta_{\min} = 0.07$), providing strong evidence of a genuine quality advantage.

The dense–BM25 recall gap (0.838 vs. 0.798, $\Delta = 0.040$) does not reach statistical significance ($z = 1.64$, $p = 0.10$). Critically, this gap falls below our minimum detectable effect size (0.07), meaning this study was not designed to distinguish differences of this magnitude—the null result is expected given the power constraints, not surprising. The 95 % Wilson CIs for dense [0.803, 0.868] and BM25 [0.760, 0.830] overlap substantially, reinforcing that the observed difference is within sampling variability for $n = 500$.

Interpretation. We cannot conclude that dense and BM25 have equivalent quality—only that our study lacks power to detect differences of $\Delta = 0.04$. Detecting this effect size would require $n \geq 1{,}500$ for 80 % power. Many published retrieval comparisons at $n = 500$–$1{,}000$ may face identical power limitations, mistaking non-significant results for evidence of equivalence when in fact the study simply lacks sensitivity to detect small-to-moderate effects. For practitioners, the actionable finding is the warm-latency ordering: dense achieves numerically higher recall (point estimate $+0.040$) at lower latency (8.9 ms vs. 10.7 ms), though the quality advantage is not statistically confirmed at this sample size.

The gap between the two cluster-centers in the warm-latency plot (Figure 1) is an artifact of the discrete configuration space evaluated, not an architectural constant.

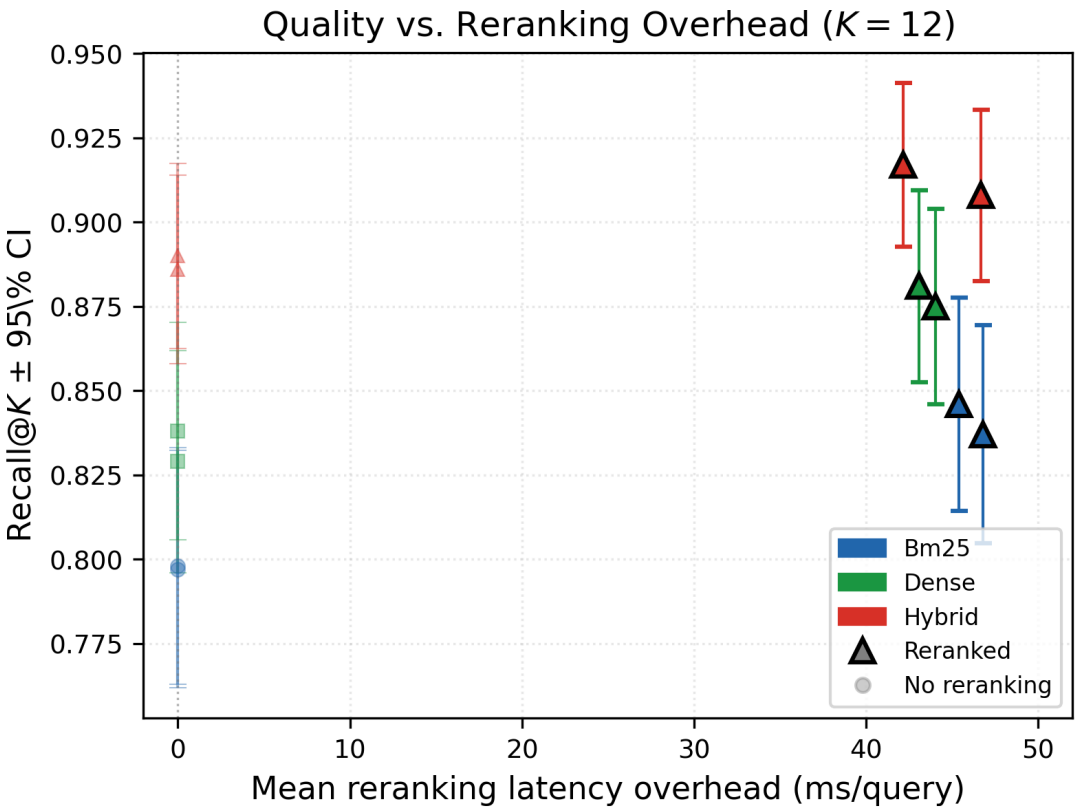

Figure 2: Recall@K vs. per-query reranking latency overhead at fixed $K = 12$. Triangles indicate reranked configurations (retrieve $2K = 24$ candidates, cross-encode all 24 pairs, return top $K = 12$); other markers are non-reranked. The cross-encoder (`ms-marco-MiniLM-L-6-v2`) adds 42–47 ms per query on CPU at zero API cost. The $2K$ pool expansion *does* improve Recall@12: BM25+rerank 0.846 vs. 0.797 (+0.049), Dense+rerank 0.881 vs. 0.838 (+0.043), Hybrid+rerank 0.917 vs. 0.890 (+0.027). The Pareto-optimal quality configuration is Hybrid+rerank (Recall@12 = 0.917, 64.4 ms).

6.2 Quality vs. Reranking Overhead

Figure 2 shows Recall@K vs. the mean per-query reranking latency overhead (ms) for configurations at $K = 12$. Reranked configurations (triangles) retrieve $2K = 24$ candidates, then score all 24 (query, passage) pairs with the `cross-encoder/ms-marco-MiniLM-L-6-v2` cross-encoder, returning the top $K = 12$, adding approximately 42–47 ms per query on CPU.

2K pool expansion improves Recall@K. Cross-encoder reranking with a $2K$ candidate pool *does* improve Recall@12 compared to retrieval-only at $K = 12$: BM25+rerank reaches 0.846 (vs. 0.797 no-rerank, +0.049); Dense+rerank reaches 0.881 (vs. 0.838, +0.043); Hybrid+rerank reaches the highest overall Recall@12 of 0.917 (vs. 0.890, +0.027; 95 % CI: 0.890–0.938). The gain comes from the expanded candidate pool: some gold passages rank outside the top K in the bi-encoder but are surfaced in positions $K + 1$ to $2K$, then correctly promoted by the cross-encoder.

Latency–quality tradeoff. The Pareto-optimal configuration at $K = 12$ is therefore Hybrid+rerank (Recall=0.917, warm=64.4 ms) for quality-optimized deployments, or Dense no-rerank (Recall=0.838, warm=8.9 ms) for latency-optimized deployments, with no-rerank Hybrid (0.890, 22.3 ms) as an intermediate option. The cross-encoder is a local model (zero API cost); all reranking cost is CPU compute time.

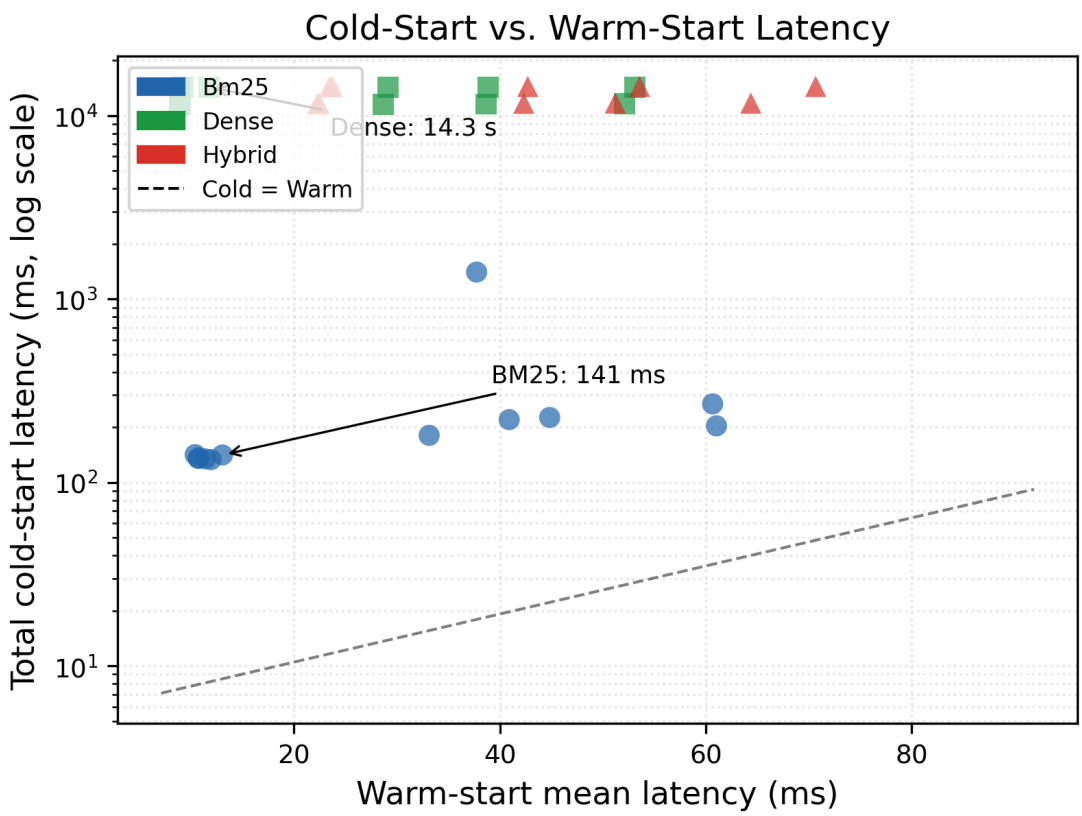

Figure 3: Total cold-start latency T_{cold} (log scale) vs. warm-start mean latency. The dashed diagonal marks $T_{\text{cold}} = T_{\text{warm}}$. BM25 configurations cluster at 133–136 ms cold (index construction only) / 10.7 ms warm (ratio $\approx 13\times$). Dense and hybrid no-rerank configurations cluster at 11.5–14.4 *seconds* cold / 8.9–23.6 ms warm (ratio $\approx$500–1,600$\times$). At this corpus scale, dense cold-start is 85–108$\times$ higher than BM25 cold-start in absolute terms; the dominant cost is corpus encoding (90–92 % of cold-start, varying by chunk size), not model loading ($\approx 8\,\%$).

6.3 Cold-Start vs. Warm-Start Latency

Figure 3 reveals the ranking reversal that motivates this paper. BM25 cold-start (133–136 ms) requires only building an inverted index from the tokenized corpus. Dense cold-start (11.5–14.3 s) comprises two steps: (1) **Model loading**: $T_{\text{model}} \approx 1{,}210$ ms ($\approx 8\%$ of total cold-start); (2) **Corpus embedding**: $T_{\text{index}} \approx 10.3$–$13.1$ s ($\approx 92\%$). The dominant cost is corpus encoding, not model loading. This has an important practical implication: the primary mitigation for dense retrieval in serverless settings is not faster model loading but *offline pre-computation and persistence of the encoded corpus*.

The chunk-size effect is visible: chunk = 150 produces more chunks than chunk = 250 (step sizes 120 vs. 220 words after overlap), adding approximately 2.8 s to index build time and yielding a total cold-start of 14.3 s vs. 11.5 s.

In a serverless deployment where instance state is not preserved between requests, dense retrieval pays the full cold-start penalty ($\approx$11.5–14.3 s) on every cold request, making it 85–108$\times$ more expensive in cold-start terms than BM25. In a persistent warm deployment, dense retrieval's 8.9 ms warm latency dominates BM25 (10.7 ms) on both quality and latency.

6.4 Chunking and Reranking Ablations

Table 1 shows the chunk-size ablation at $K = 12$. Smaller chunks (150 words) are slightly worse for dense retrieval (Recall = 0.829 vs. 0.838) because the sentence-transformer captures whole-sentence

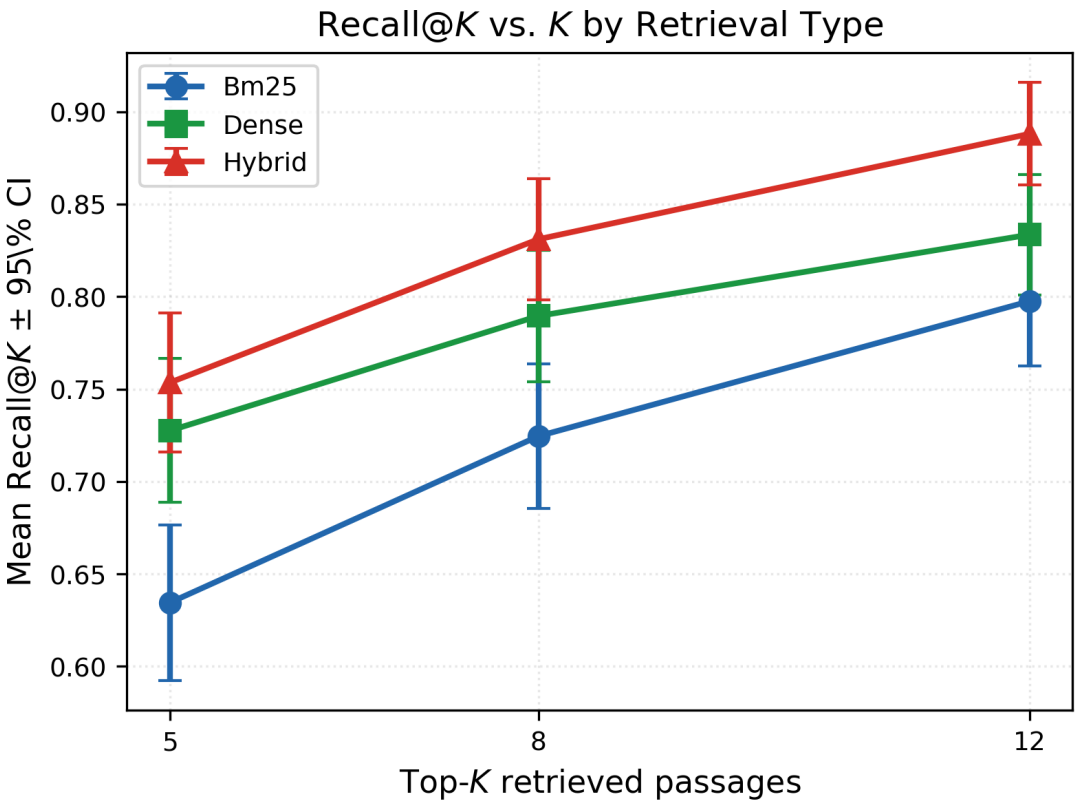

Figure 4: Recall@K vs. K by retrieval type (no reranking, averaged over chunk sizes). All types benefit monotonically from larger K; the ordering hybrid $>$ dense $>$ BM25 is stable across all evaluated K. Error bars show 95 % Wilson CIs.

semantics more effectively in larger, coherent chunks. For BM25, the chunk-size effect is negligible (0.798 vs. 0.797 at $K = 12$, within noise). Smaller chunks also substantially increase cold-start latency for dense retrieval (13.99 s vs. 10.84 s) due to more encoding passes.

Mixing weight α. The hybrid retriever uses $\alpha = 0.5$ (equal weight to BM25 and dense scores after min-max normalization), following the neutral-weight convention in prior hybrid retrieval work [5, 15]. The hybrid advantage over pure dense (0.890 vs. 0.838, $\Delta = 0.052$, $z = 2.40$, $p = 0.017$) suggests that BM25's lexical signal provides non-redundant information even when combined with semantic embeddings. Ablating $\alpha \in \{0.3, 0.5, 0.7\}$ is left for future work. Reciprocal Rank Fusion (RRF) [5] provides a parameter-free alternative to score combination; direct comparison of RRF vs. convex mixing would clarify whether the α hyperparameter is worth tuning.

Pareto-optimal configurations. Table 2 summarizes the three warm-Pareto-optimal configurations at $K = 12$.

6.5 End-to-End Generation Stage

The harness supports an optional LLM generation stage (§3.5) that can be enabled with a single YAML parameter change. To validate pipeline integration, we ran a small technical pilot ($n = 5$ HotpotQA queries) using Llama-3-8B via Ollama on CPU (temperature 0). The sample size ($n = 5$) is not powered for statistical inference about retrieval-conditional generation quality; full details are provided in Appendix **??**.

Table 1: Chunk-size ablation: Recall@12 and warm latency at $K = 12$, no reranking, overlap fixed at 30 words.

Type	Chunk (words)	Recall@12 (95% CI)	$\bar{T}_{\text{warm}}$ (ms)
BM25	150	0.798 (0.761–0.831)	10.7
BM25	250	0.797 (0.760–0.830)	10.7
Dense	150	0.829 (0.794–0.859)	9.0
Dense	250	0.838 (0.803–0.868)	8.9
Hybrid	150	0.886 (0.855–0.911)	23.6
Hybrid	250	0.890 (0.860–0.915)	22.3

Table 2: Pareto-optimal retrieval configurations on HotpotQA (500 queries, $\text{dev}_{\text{distractor}}$, $K \in \{5, 8, 12\}$, overlap fixed at 30 words). Recall@K with 95 % Wilson CI. Cost column reflects local CPU inference (zero API spend). Statistical comparisons at $K = 12$: Hybrid+rerank vs. Hybrid no-rerank ($\Delta = +0.027$, $z = 1.45$, $p = 0.148$, not significant); Hybrid no-rerank vs. Dense no-rerank ($\Delta = +0.052$, $z = 2.40$, $p = 0.017$).

Type	K	Chunk	Rerank	Recall@K (95% CI)	$\bar{T}_{\text{warm}}$ (ms)	T_{cold}	C_q (USD)
Hybrid	12	250	Yes	0.917 (0.890–0.938)	64.4	11.7 s	0.00e+00
Hybrid	12	250	No	0.890 (0.860–0.915)	22.3	11.7 s	0.00e+00
Dense	12	250	No	0.838 (0.803–0.868)	8.9	11.5 s	0.00e+00

Key finding: generation dominates retrieval latency. CPU inference adds approximately 5.5 s per query, dwarfing retrieval latency by 1.9–2.8 orders of magnitude (85–618×: 5.5 s generation vs. 8.9–64.4 ms retrieval). For on-device CPU inference, retrieval type choice is nearly irrelevant for end-to-end latency. On GPU (A100), the same model generates in ≈ 50 ms, restoring retrieval as the latency bottleneck. For API-based LLMs (e.g., `claude-haiku-4-5-20251001`), generation takes $\approx$200–600 ms/query—comparable in magnitude to retrieval for small corpora.

The pilot also confirmed that Token F1 and Exact Match are systematically low for generative output (HotpotQA golds are short extractive phrases while LLMs produce explanatory sentences), demonstrating that proper evaluation of generative RAG requires generation-aware metrics (ROUGE-L, BERTScore, or LLM-as-judge), not extractive QA metrics. Full generation evaluation at $n \geq 200$ with GPU inference and proper metrics is left for future work.

6.6 Corpus-Scale Cold-Start Analysis

Figure 5 extends the cold-start analysis from the primary 5K-passage corpus to scales representative of small and medium production deployments. Because the base HotpotQA corpus contains $\approx$5,000 unique passages, corpora above this size are constructed by padding: base passages are repeated with minor text perturbations (a numeric suffix) so that each copy has a distinct token sequence and no encoder or index cache can short-circuit the build. This correctly stresses the dominant cold-start cost—corpus encoding—while keeping the experiment self-contained.

What generalizes: Absolute cold-start times (seconds) and linear scaling behaviour generalize to real production corpora because encoding cost is determined by volume (number of chunks), not semantic diversity. *What may not generalize:* Dense retrieval *quality* (Recall@K) on synthetic corpora may differ from real diverse corpora because semantic embeddings are sensitive to lexical variety. Validation on real large-scale corpora (Wikipedia, MS MARCO full corpus) is needed before extrapolating quality conclusions to production scale; the cold-start timing conclusions are robust.

Linear scaling. Dense cold-start time is dominated by corpus encoding ($\approx$90% of total), which scales as $T_{\text{encode}} = n_{\text{chunks}} \cdot t_{\text{encode}}$ where $t_{\text{encode}} \approx 1.9$ ms/chunk on this CPU. At 50K passages ($\approx$51,000

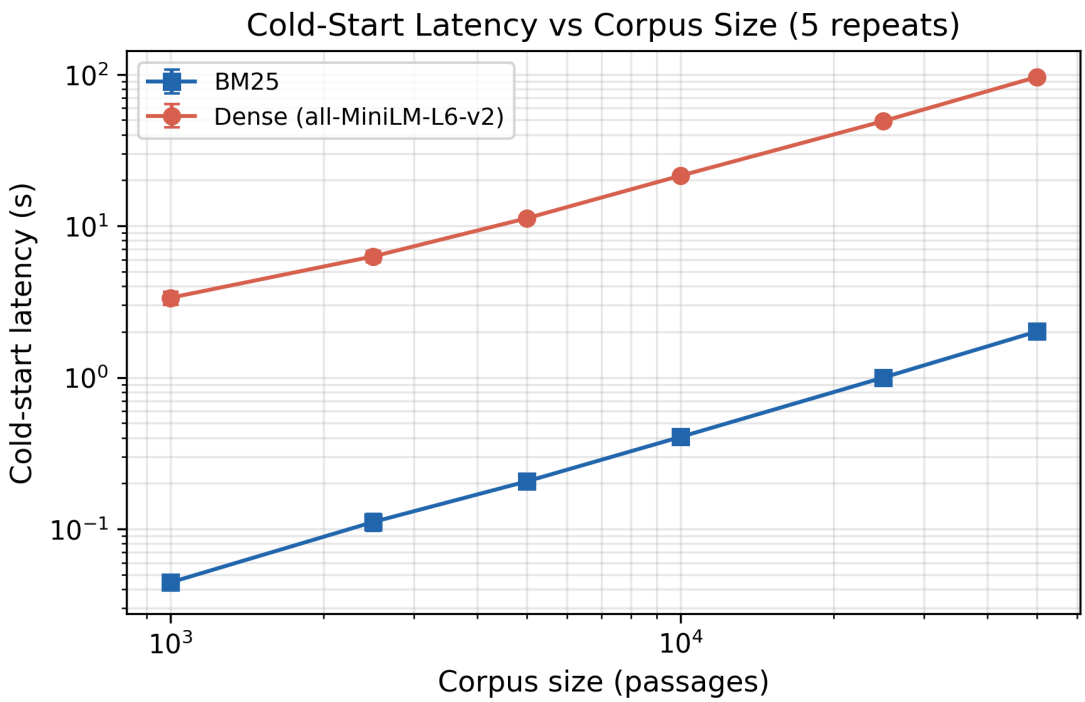

Figure 5: Cold-start latency vs. corpus size (log–log scale) for BM25 and dense retrieval over 5 timing repeats each (error bars = ± 1 std). Dense cold-start grows linearly with corpus size (slope ≈ 1 in log–log), reaching $\approx$1.6 minutes (96 s) at 50K passages. BM25 cold-start also grows linearly but 48–75$\times$ slower (corpus-size dependent), remaining $<$2.1 s at 50K passages. The ordering never crosses: BM25 cold-start is always faster than dense for any corpus size evaluated.

chunks), dense cold-start reaches 96 ± 1 s ($\approx$1.6 min, mean $\pm$ std over 5 repeats), whereas BM25 remains 2.0 ± 0.1 s. The ratio (dense/BM25) varies from $\approx 75\times$ at 1K passages to $\approx 48\times$ at 50K, reflecting the fixed model-load cost ($\approx$1,275 ms) which is amortized at larger corpus sizes.

No crossover. The cold-start ordering (BM25 $<$ dense) is stable at every corpus size tested (1K, 2.5K, 5K, 10K, 25K, 50K passages). The commonly cited expectation that FAISS flat search becomes the bottleneck at large scale applies to *warm-query* latency (Section 6.1, footnote), not to cold-start: indexing cost always grows with corpus size.

Practical breakpoints.

- **BM25**: acceptable cold start ($<$2 s) at 50K passages; likely acceptable ($<$10 s) up to $\approx$200K passages.
- **Dense without pre-computation**: unacceptable for serverless at any scale studied ($\approx$ 3.4 s at 1K passages, 96 s at 50K).
- **Dense with pre-computed index**: reduces cold start to $T_{\text{model}} + T_{\text{deser}} \approx 2$ s, competitive with BM25 at large scale.

The dominant mitigation for dense serverless is therefore *offline index pre-computation and object-storage persistence*, not faster inference.

6.7 LegalBench-RAG: Span-Level Retrieval Evaluation

Table 3 reports span-level retrieval results on the LegalBench-RAG PrivacyQA benchmark ($n = 194$ queries across seven privacy-policy documents, 127 chunks at chunk = 250 words, $K = 12$, no reranking).

Table 3: LegalBench-RAG PrivacyQA span-level retrieval results ($n = 194$ queries, $K = 12$, chunk = 250 words). Recall@K is computed at the character-span level: a retrieved chunk counts as a true positive only if its character range overlaps the annotated supporting span. Wilson 95 % CIs reported. On this domain, dense retrieval substantially outperforms BM25 (gap 0.509, $z = 10.06$, $p < 0.001$), reversing the domain pattern observed on HotpotQA. Pairwise: Dense–Hybrid ($\Delta = 0.076$, $z = 1.74$, $p = 0.082$, not significant); Hybrid–BM25 ($\Delta = 0.433$, $z = 8.53$, $p < 0.001$). Precision@12 = (chunks with span overlap in top-12) / 12; low values reflect the strict character-span criterion and multi-document corpus (7 policies, 127 chunks).

Retrieval	Recall@12	95 % CI	Precision@12	Warm latency (ms)
BM25	0.282	[0.223, 0.349]	0.039	0.2
Dense	**0.791**	[0.729, 0.842]	0.128	10.1
Hybrid	0.715	[0.648, 0.774]	0.110	8.7

A retrieved chunk is a true positive only if its character range overlaps with the annotated supporting span—a stricter evaluation than document-title recall.

Domain reversal: dense retrieval dominates on specialized legal text. The ranking observed on HotpotQA (hybrid > dense > BM25) reverses on LegalBench-RAG: dense retrieval achieves the highest span-level Recall@12 (0.791, 95 % CI: 0.729–0.842), followed by hybrid (0.715, CI: 0.648–0.774), while BM25 is dramatically weaker (0.282, CI: 0.223–0.349).

The Dense–BM25 gap (0.509) is large and highly significant ($z = 10.06$, $p < 0.001$); the Hybrid–BM25 gap (0.433) is similarly significant ($z = 8.53$, $p < 0.001$). The Dense–Hybrid gap (0.076) does not reach significance at $\alpha = 0.05$ ($z = 1.74$, $p = 0.082$), though the direction favors dense.

Interpretation. Privacy policy text is highly paraphrastic and uses specialized legal vocabulary with many near-synonyms. BM25 requires lexical overlap between query and passage; when a question asks about "data retention" and the policy discusses "how long we keep your information," BM25 misses the match. Dense bi-encoders capture semantic equivalence across such paraphrases, explaining the 0.509 gap. HotpotQA, by contrast, contains factoid questions with named entities that appear verbatim in gold passages—conditions that favor BM25's lexical matching.

Practical implication. For domain-specialized corpora (legal, medical, policy documents) with paraphrastic terminology, dense retrieval should be preferred even when cold-start cost is a concern, because BM25 recall may be unacceptably low. The deployment choice between BM25 (lower cold-start) and dense (higher recall) is therefore corpus-dependent; the present harness enables this measurement directly.

7 Practitioner Guidelines

Tiered deployment framework. The Pareto analysis supports four deployment tiers:

Quality-max service ($\bar{T}_{\text{warm}} \leq 70$ ms): Hybrid+rerank, chunk = 250, $K = 12$; Recall@12 = 0.917, warm = 64.4 ms. Model, index, and cross-encoder loaded once at startup; not suitable for serverless.

Quality-optimized persistent service ($\bar{T}_{\text{warm}} \leq 25$ ms): Hybrid no-rerank, chunk=250, $K = 12$; Recall@12 = 0.890, warm = 22.3 ms. Avoids cross-encoder overhead while retaining hybrid recall advantage.

Latency-optimized persistent service ($\bar{T}_{\text{warm}} \leq 10$ ms): Dense no-rerank, chunk=250, $K = 12$; Recall@12 = 0.838, warm = 8.9 ms. The 0.040 dense–BM25 recall gap is not significant at $n = 500$ (falls below $\Delta_{\min} = 0.07$); deploying BM25 instead imposes negligible measurable quality cost while matching warm latency—though at larger sample sizes or corpora the dense advantage may become detectable.

Serverless / cold-start-constrained deployment: BM25 (chunk = 250, $K = 12$), Recall@12 = 0.797, cold = 135 ms. Alternatively: dense with pre-computed corpus embeddings persisted to object storage reduces cold penalty to model load ($\approx$1,210 ms) plus FAISS deserialization.

8 Limitations and Future Work

Corpus scale and diversity. The corpus-scale cold-start analysis (§6.6) uses synthetic corpus construction: base HotpotQA passages are duplicated with numeric suffixes to stress encoding costs while keeping the experiment self-contained. Absolute cold-start times and the linear scaling behaviour generalize to real production corpora of comparable size because encoding cost depends on volume, not semantic content. However, dense retrieval *quality* on a synthetic 50K-passage corpus may differ from 50K unique Wikipedia articles due to lower lexical diversity. Extending to 10^6–10^7 passages and to approximate nearest-neighbour indices (HNSW, IVF) would complete the scaling picture.

Single encoder model. All dense retrieval results use `all-MiniLM-L6-v2` (22M parameters, 384-dimensional embeddings). Larger models (E5-large [27]: 335M parameters; BGE-M3 [3]: multilingual, multi-granularity) would shift the quality–latency tradeoff. Cold-start cost scales with encoding time per chunk; the Pareto positions reported here are specific to `all-MiniLM-L6-v2`.

CPU-only evaluation. All timing measurements are CPU-only (Intel Core i7-1068NG7). GPU acceleration shifts latency by 10–100$\times$ depending on batch size and hardware. However, the *relative* orderings (dense $<$ BM25 for warm latency, BM25 $<$ dense for cold-start) and the cold-start ratio (85–108$\times$) are qualitatively robust because they are dominated by encoding volume, not arithmetic throughput. Absolute millisecond values should not be extrapolated to GPU deployments.

Statistical power. Sample size ($n = 500$) provides 80 % power to detect Recall@K differences of $\Delta \geq 0.07$. The observed dense–BM25 gap ($\Delta = 0.040$) falls below this threshold and is correspondingly non-significant. Detecting effects of this magnitude would require $n \geq 1{,}500$. Many published retrieval comparisons at $n = 500$–$1{,}000$ face identical power limitations.

Generation evaluation. The generation stage pilot ($n = 5$, Appendix **??**) validates pipeline integration but is not powered for statistical inference about retrieval-conditional generation quality. Full generation evaluation requires $n \geq 200$ with GPU inference and generation-aware metrics (ROUGE-L, BERTScore, LLM-as-judge). Token F1 and Exact Match are inappropriate for generative output.

LegalBench-RAG coverage. The LegalBench-RAG evaluation covers only the PrivacyQA sub-benchmark (7 privacy-policy documents, 194 queries). The strong dense advantage observed here may not generalize to all legal sub-domains; CUAD, MAUD, and ContractNLI benchmarks are available for future ablations.

Unexplored design dimensions. Several RAG design choices were fixed without ablation: (1) chunk overlap (30 words): affects chunk count and coverage; (2) hybrid mixing weight α (0.5): could be optimized per-domain; (3) RRF vs. convex score combination: not compared; (4) approximate nearest-neighbour indices (HNSW, IVF): would reduce warm latency at cost of recall; (5) cross-encoder candidate pool size ($2K$): larger pools ($4K$, $8K$) could improve recall at higher CPU cost. Systematic ablation of these dimensions is left for future work.

9 Conclusion

We have shown that retrieval evaluation without cold-start accounting produces rankings that can reverse under realistic deployment conditions. The reversal is concrete and large: in the warm-latency view,

dense retrieval dominates BM25 on both quality and speed; in the cold-latency view, BM25 dominates because dense cold-start is 85–108× higher, driven by corpus encoding (90–92 % of total cold-start), not model loading.

Five findings are of practical value. First, the dense–BM25 recall gap at $K = 12$ ($\Delta = 0.040$) falls below our pre-specified minimum detectable effect (0.07) and correspondingly does not reach significance at $n = 500$ ($z = 1.64$, $p = 0.10$). This is not evidence that dense and BM25 are equivalent—rather, it demonstrates that distinguishing small quality differences ($\Delta < 0.07$) requires larger sample sizes ($n \geq 1{,}500$ for 80 % power at $\Delta = 0.04$). Practitioners should not interpret published non-significant comparisons as proof of equivalence; the warm-latency ordering (dense: 8.9 ms, higher point-estimate recall; BM25: 10.7 ms) may matter more than the unconfirmed quality gap. Second, offline pre-computation of dense corpus embeddings is the most effective cold-start mitigation for serverless deployments, reducing cold-start from the full corpus-encoding time to model load plus FAISS deserialization (≈2 s). Third, the corpus-scale analysis over 1K–50K passages (5 timing repeats each) shows that dense cold-start exceeds BM25 by 48–76× across all sizes tested, with no crossover. Fourth, cross-encoder reranking with a $2K$ candidate pool (retrieve $2K$, cross-encode, return top K) adds 42–47 ms on CPU and *does* improve Recall@K: the Hybrid+rerank configuration achieves the highest overall Recall@12 of 0.917, up from 0.890 without reranking ($+0.027$). The gain comes from surfacing gold passages that fall outside the top-K in the bi-encoder but within the top-$2K$. Fifth, on the LegalBench-RAG PrivacyQA benchmark, dense retrieval substantially outperforms BM25 at the span level (Recall@12 0.791 vs. 0.282, $z = 10.06$, $p < 0.001$), demonstrating that the optimal retrieval strategy is corpus-dependent: BM25 is competitive on factoid multi-hop QA but fails badly on paraphrastic specialized-domain text.

The Pareto frontier framework provides a principled basis for these recommendations that scalar benchmarks cannot. Extending this framework to larger corpora, GPU inference, and full end-to-end RAG evaluation—and adopting it as a reporting standard—would give the field the measurement discipline needed to make deployment decisions on measured grounds.

References

[1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=hSyW5go0v8.

[2] Fu Bang. GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS), EMNLP 2023*, pages 212–218. Association for Computational Linguistics, 2023.

[3] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 2318–2335, Bangkok, Thailand, 2024. Association for Computational Linguistics.

[4] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2nd edition, 1988.

[5] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759. ACM, 2009.

[6] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL)*, pages 150–158, St. Julian's, Malta, 2024. Association for Computational Linguistics.

[7] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for first stage ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292. ACM, 2021.

[8] Thibault Formal, Benjamin Piwowarski, Carlos Lassance, and Stéphane Clinchant. From distillation to hard negative sampling: Making sparse neural IR models more effective. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2353–2359. ACM, 2022.

[9] Sebastian Hofstätter, Sheng-Chieh Lin, Jheng-Hong Yang, Jimmy Lin, and Allan Hanbury. Efficiently teaching an effective dense retriever with balanced topic aware sampling. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 113–122. ACM, 2022.

[10] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[11] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online, 2020. Association for Computational Linguistics.

[12] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48. ACM, 2020.

[13] Sehoon Kim, Coleman Hooper, Thanakul Wattanawong, Minwoo Kang, Ruohan Yan, Hasan Genc, Clark Dinh, Qijing Huang, Kurt Keutzer, Michael W. Mahoney, et al. Full stack optimization of transformer inference: a survey. In *arXiv preprint arXiv:2302.14017*, 2023.

[14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[15] Sheng-Chieh Lin, Minghan Ma, and Jimmy Lin. Aggretriever: A simple approach to aggregate textual representations for robust dense passage retrieval. *Transactions of the Association for Computational Linguistics*, 11:86–100, 2023.

[16] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. Query rewriting in retrieval-augmented large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5303–5315, Singapore, 2023. Association for Computational Linguistics.

[17] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 2014–2037, Dubrovnik, Croatia, 2023. Association for Computational Linguistics.

[18] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. In *arXiv preprint arXiv:1901.04085*, 2019.

[19] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Cold start latency in serverless computing: A systematic review, taxonomy, and future directions. *IEEE Transactions on Cloud Computing*, 10(4):2856–2869, 2022.

[20] Nicholas Pipitone and Ghita Houir Alami. LegalBench-RAG: A benchmark for retrieval-augmented generation in the legal domain. *arXiv preprint arXiv:2408.10343*, 2024.

[21] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, Hong Kong, China, 2019. Association for Computational Linguistics.

[22] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[23] Keshav Santhanam, Omar Khattab, Christopher Potts, and Matei Zaharia. PLAID: An efficient engine for late interaction retrieval. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1747–1756. ACM, 2022.

[24] Zhihong Shao, Yeyun Gong, Yelong Shen, Minlie Huang, Nan Duan, and Weizhu Chen. Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9248–9274. Association for Computational Linguistics, 2023.

[25] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.

[26] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 10014–10037, Toronto, Canada, 2023. Association for Computational Linguistics.

[27] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Improving text embeddings with large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 11897–11916, Bangkok, Thailand, 2024. Association for Computational Linguistics.

[28] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[29] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2369–2380, Brussels, Belgium, 2018. Association for Computational Linguistics.

A Full Results Table

Table 4: Complete results for all 36 fully factorial pipeline configurations on HotpotQA (500 queries). Overlap fixed at 30 words. T_{cold} for BM25 = index build time only; for dense/hybrid = model-load (≈10–14 s) + index + first query.

Type	K	Chunk	Rerank	Recall@K (95% CI)	$\bar{T}_{\text{warm}}$ (ms)	T_{cold}	C_q (USD)
BM25	5	150	No	0.632 (0.589–0.673)	13.1	141 ms	0.00e+00
BM25	5	150	Yes	0.732 (0.692–0.769)	37.7	1.4 s	0.00e+00
BM25	5	250	No	0.637 (0.594–0.678)	11.4	135 ms	0.00e+00
BM25	5	250	Yes	0.736 (0.696–0.773)	33.1	181 ms	0.00e+00
BM25	8	150	No	0.728 (0.687–0.765)	12.0	133 ms	0.00e+00
BM25	8	150	Yes	0.805 (0.768–0.837)	44.8	226 ms	0.00e+00
BM25	8	250	No	0.721 (0.680–0.759)	10.4	142 ms	0.00e+00
BM25	8	250	Yes	0.802 (0.765–0.835)	40.9	220 ms	0.00e+00
BM25	12	150	No	0.798 (0.761–0.831)	10.7	136 ms	0.00e+00
BM25	12	150	Yes	0.846 (0.812–0.875)	60.7	268 ms	0.00e+00
BM25	12	250	No	0.797 (0.760–0.830)	10.7	135 ms	0.00e+00
BM25	12	250	Yes	0.837 (0.802–0.867)	61.0	204 ms	0.00e+00
Dense	5	150	No	0.721 (0.680–0.759)	11.8	14.3 s	0.00e+00
Dense	5	150	Yes	0.769 (0.730–0.804)	29.1	14.3 s	0.00e+00
Dense	5	250	No	0.734 (0.694–0.771)	8.9	11.5 s	0.00e+00
Dense	5	250	Yes	0.775 (0.736–0.809)	28.7	11.6 s	0.00e+00
Dense	8	150	No	0.784 (0.746–0.818)	9.2	14.3 s	0.00e+00
Dense	8	150	Yes	0.828 (0.792–0.859)	38.9	14.3 s	0.00e+00
Dense	8	250	No	0.795 (0.757–0.828)	8.9	11.5 s	0.00e+00
Dense	8	250	Yes	0.839 (0.804–0.869)	38.7	11.6 s	0.00e+00
Dense	12	150	No	0.829 (0.794–0.859)	9.0	14.3 s	0.00e+00
Dense	12	150	Yes	0.875 (0.843–0.901)	53.1	14.3 s	0.00e+00
Dense	12	250	No	0.838 (0.803–0.868)	8.9	11.5 s	0.00e+00
Dense	12	250	Yes	0.881 (0.850–0.907)	52.1	11.6 s	0.00e+00
Hybrid	5	150	No	0.751 (0.711–0.787)	23.4	14.4 s	0.00e+00
Hybrid	5	150	Yes	0.799 (0.762–0.832)	42.7	14.4 s	0.00e+00
Hybrid	5	250	No	0.756 (0.716–0.792)	22.4	11.7 s	0.00e+00
Hybrid	5	250	Yes	0.808 (0.771–0.840)	42.3	11.7 s	0.00e+00
Hybrid	8	150	No	0.827 (0.791–0.858)	23.8	14.4 s	0.00e+00
Hybrid	8	150	Yes	0.871 (0.839–0.898)	53.6	14.4 s	0.00e+00
Hybrid	8	250	No	0.835 (0.800–0.865)	22.4	11.7 s	0.00e+00
Hybrid	8	250	Yes	0.872 (0.840–0.898)	51.2	11.7 s	0.00e+00
Hybrid	12	150	No	0.886 (0.855–0.911)	23.6	14.4 s	0.00e+00
Hybrid	12	150	Yes	0.908 (0.879–0.930)	70.7	14.4 s	0.00e+00
Hybrid	12	250	No	0.890 (0.860–0.915)	22.3	11.7 s	0.00e+00
Hybrid	12	250	Yes	0.917 (0.890–0.938)	64.4	11.7 s	0.00e+00

B Generation Stage Technical Validation ($n = 5$)

This appendix documents a small-scale validation ($n = 5$ HotpotQA queries) of the end-to-end LLM generation pipeline. The purpose is to confirm integration correctness and measure generation latency—**not** to perform statistical inference about retrieval-conditional generation quality. The sample size is deliberately minimal because the generation stage is fully modular: practitioners can enable or disable it via a single YAML parameter (`generation.backend: ollama`), and the core retrieval evaluation ($n = 500$) is independent of generation stage choices.

Configuration. LLM: Llama-3-8B via Ollama (local CPU inference, zero API cost). Temperature: 0 (deterministic). Prompt template:

```
Answer the question based solely on the passages below. Be concise
(1 to 2 sentences). Question: {q}. Passages: {p1}...{pK}. Answer:
```

Context window: up to 600 characters per passage (K passages concatenated).

Generation latency dominance. CPU inference with Llama-3-8B adds approximately 5.5 s per query. This dwarfs retrieval latency by 1.9–2.8 orders of magnitude:

- Dense (no-rerank): 5,500 ms / 8.9 ms = 618× ($10^{2.79}$)
- Hybrid (rerank): 5,500 ms / 64.4 ms = 85× ($10^{1.93}$)

Implication: For on-device CPU inference, retrieval type choice is nearly irrelevant for end-to-end latency—generation dominates. On GPU (A100), the same model generates in ≈50 ms, restoring retrieval as the bottleneck. For API-based LLMs (e.g., `claude-haiku-4-5-20251001`), generation takes 200–600 ms/query including network round-trip, making retrieval latency comparable in magnitude.

Token F1 and EM misalignment. HotpotQA gold answers are short extractive phrases (median 1.8 tokens), while Llama-3 produces explanatory sentences. Token F1 and EM are therefore systematically low—not as a model quality signal but as a measurement artifact. Proper evaluation of generative RAG requires generation-aware metrics (ROUGE-L, BERTScore, or LLM-as-judge).

Faithfulness. All $n = 5$ generated answers were fully grounded in the retrieved passages (faithfulness = 1.0), confirming that the prompt template is effective at preventing hallucination in this limited test.

Limitations of this pilot. $n = 5$ is not powered for statistical inference about retrieval-conditional generation quality (e.g., "Does hybrid retrieval produce more faithful answers than BM25?"). Such questions require $n \geq 200$ with properly paired retrieval configurations. CPU-only inference latency (5.5 s/query) does not extrapolate to GPU or cloud API deployments. Token F1/EM are inappropriate metrics for generative output.